

01_corpus_descriptive_report

May 7, 2026

1 01 Corpus Descriptive Report

This notebook is a descriptive reporting and validation layer for the prepared AfD-project corpus data.

It reads prepared datasets from data/ and does not rebuild retrieval/cleaning/deduplication/pipeline stages.

Scope of this notebook: - descriptive corpus inspection - monthly coverage/integrity checks - simple anomaly/duplicate burden review - first simple monthly media salience indicator

Analytical rule used here: - all articles = audit/QC universe - kept articles = analytical corpus

```
[1]: from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib
try:
    from IPython import get_ipython
    _ip = get_ipython()
    if _ip is not None:
        _ip.run_line_magic("matplotlib", "inline")
    else:
        matplotlib.use("Agg")
except Exception:
    matplotlib.use("Agg")
import matplotlib.pyplot as plt

pd.set_option("display.max_columns", 120)
pd.set_option("display.width", 140)

PROJECT_ROOT = Path("..").resolve()
DATA_DIR = PROJECT_ROOT / "data"
FIGURES_DIR = PROJECT_ROOT / "figures" / "01_notebook_figures"
OUTPUT_TABLE_DIR = DATA_DIR / "descriptive_tables"

MASTER_PATH = DATA_DIR / "master_articles.parquet"
MONTHLY_PATH = DATA_DIR / "monthly_summary.parquet"
```

```

FIGURES_DIR.mkdir(parents=True, exist_ok=True)
OUTPUT_TABLE_DIR.mkdir(parents=True, exist_ok=True)

print("PROJECT_ROOT:", PROJECT_ROOT)
print("MASTER_PATH exists:", MASTER_PATH.exists())
print("MONTHLY_PATH exists:", MONTHLY_PATH.exists())

```

```

PROJECT_ROOT: C:\PythonProjects\AfD-project
MASTER_PATH exists: True
MONTHLY_PATH exists: True

```

```

[2]: articles_all = pd.read_parquet(MASTER_PATH)
      monthly = pd.read_parquet(MONTHLY_PATH)

      if "keep_drop_review" in articles_all.columns:
          _kdr = articles_all["keep_drop_review"].astype(str).str.strip().str.lower()
          articles_kept = articles_all[_kdr == "keep"].copy()
      else:
          raise ValueError("keep_drop_review column not found in master_articles.
                              ↳parquet")

      print("articles_all shape:", articles_all.shape)
      print("articles_kept shape:", articles_kept.shape)
      print("monthly shape:", monthly.shape)

      display(articles_all.head(3))
      display(monthly.head(3))

```

```

articles_all shape: (10569, 35)
articles_kept shape: (8882, 35)
monthly shape: (156, 32)

```

	source_archive	date	year	month	article_id	corpus_type	corpus_name	batch_id
0	87b4fbb8d229a88fca9b075d3f1d67cc37ad98f7	2013-01-31	2013	2013-01	main_monthly	Main Corpus	2013-01	
	↳2013-01(42).ZIP							
1	5c542629da2012abfe225f39ad7d5812a1c34d12	2013-01-15	2013	2013-01	main_monthly	Main Corpus	2013-01	
	↳2013-01(42).ZIP							
2	0c964dd4545a5c14b3d36283db4da1270ef88a80	2013-01-18	2013	2013-01	main_monthly	Main Corpus	2013-01	
	↳2013-01(42).ZIP							

	publication	publication_normalized
	title_original	↳
0	Der Tagesspiegel	der tagesspiegel
	↳Flüchtlinge Fast 60...	
1	Die Welt	die welt
	↳Deutschland Di...	

```

2 Rheinische Post          rheinische post          "Ich will in meiner
↳ Heimat wählen"

                                title_normalized          byline          □
↳                                section word_count body_char_count \
0 15 milliarden für syriens flüchtlinge fast 60 ...          POLITIK;□
↳ S. 5; Ausg. 21588          167          1223
1 40 prozent mehr asylbewerber in deutschland di...          Manuel Bewarder          □
↳ POLITIK; S. 5; Ausg. 12          710          5058
2                                ich will in meiner heimat wählen Von Susanne Genath          □
↳                                600          3907

                                text_body          □
↳ body_hash_exact duplicate_group_id duplicate_category \
0 Istanbul/Kairo - Mit rund anderthalb Milliarde... □
↳ 37aad556cc2dcaa1b066821381b597f45c4f2af3          unique
1 Die Zahl der Asylbewerber in Deutschland ist i... □
↳ 8dafdceb36b87384fec9808055b70357d430877e          unique
2 Güler Ayranci hat nur noch den deutschen Pass... □
↳ f6401f504217028080afb5e3724bc926416f624a          unique

keep_drop_review drop_reason kept_representative_of_group□
↳ representative_selection_reason malformed_flag malformed_reason \
0                keep                no                □
↳                no
1                keep                no                □
↳                no
2                keep                no                □
↳                no

                                source_file_name          □
↳                                source_file_path protocol_version notes \
0 1,5 Milliarden für Syriens Flüchtlinge Fast 60... C:
↳ \PythonProjects\AfD-project\Corpora\Main Cor...          v2
1 40 Prozent mehr Asylbewerber in Deutschland Di... C:
↳ \PythonProjects\AfD-project\Corpora\Main Cor...          v2
2                _Ich will in meiner Heimat wählen_.PDF C:
↳ \PythonProjects\AfD-project\Corpora\Main Cor...          v2

query_version source_basket_version language publication_type          □
↳ date_raw
0                de                news    Donnerstag 31.□
↳ Januar 2013
1                de                news    Dienstag 15.□
↳ Januar 2013

```

```

2                                de                news        18. Januar
↳2013 Freitag

  month_id year month batch_id raw_article_count kept_article_count
↳dropped_article_count keep_share unique_source_count \
0 2013-01 2013      1 2013-01                42                38
↳                4      0.904762                8
1 2013-02 2013      2 2013-02                55                41
↳                14      0.745455                8
2 2013-03 2013      3 2013-03                67                61
↳                6      0.910448                7

  total_word_count_kept mean_word_count_kept median_word_count_kept
↳duplicate_exact_count duplicate_near_count \
0                17346                456.474                342.0
↳                1                0
1                20616                502.829                451.0
↳                2                0
2                32763                537.098                479.0
↳                0                0

  regional_variant_count malformed_count reader_letter_drop_count
↳commentary_noncore_drop_count very_short_low_value_drop_count \
0                1                0                1
↳                1                0
1                4                0                0
↳                8                0
2                2                0                0
↳                4                0

  source_top1 source_top1_count source_top2
↳source_top2_count source_top3 source_top3_count freeze_status \
0      Rheinische Post                10 taz, die tageszeitung
↳      9      Die Welt                8      frozen
1 taz, die tageszeitung                11      Berliner Zeitung
↳      6      Der Tagesspiegel                6      frozen
2 taz, die tageszeitung                24      Die Welt
↳      12      Der Tagesspiegel                11      frozen

  notes kept_share_of_raw duplicate_burden dropped_share shock_month_flag
↳shock_window_flag_placeholder
0                0.904762                0.047619                0.095238                0
1                0.745455                0.109091                0.254545                0
2                0.910448                0.029851                0.089552                0

```

```
[3]: print("Article columns:")
print(list(articles_all.columns))
print()
print("Monthly columns:")
print(list(monthly.columns))

article_date_like = [c for c in articles_all.columns if any(k in c.lower() for
↳ k in ["date", "month", "year"])]
monthly_date_like = [c for c in monthly.columns if any(k in c.lower() for k in
↳ ["date", "month", "year"])]

print()
print("Article date/month-like columns:", article_date_like)
print("Monthly date/month-like columns:", monthly_date_like)

if "article_id" in articles_all.columns:
    dup_article_id_rows = int(articles_all["article_id"].duplicated().sum())
    print("Duplicate article_id row count (all rows):", dup_article_id_rows)
else:
    print("article_id not found; skipping uniqueness check")
```

Article columns:

```
['article_id', 'corpus_type', 'corpus_name', 'batch_id', 'source_archive',
'date', 'year', 'month', 'publication', 'publication_normalized',
'title_original', 'title_normalized', 'byline', 'section', 'word_count',
'body_char_count', 'text_body', 'body_hash_exact', 'duplicate_group_id',
'duplicate_category', 'keep_drop_review', 'drop_reason',
'kept_representative_of_group', 'representative_selection_reason',
'malformed_flag', 'malformed_reason', 'source_file_name', 'source_file_path',
'protocol_version', 'notes', 'query_version', 'source_basket_version',
'language', 'publication_type', 'date_raw']
```

Monthly columns:

```
['month_id', 'year', 'month', 'batch_id', 'raw_article_count',
'kept_article_count', 'dropped_article_count', 'keep_share',
'unique_source_count', 'total_word_count_kept', 'mean_word_count_kept',
'median_word_count_kept', 'duplicate_exact_count', 'duplicate_near_count',
'regional_variant_count', 'malformed_count', 'reader_letter_drop_count',
'commentary_noncore_drop_count', 'very_short_low_value_drop_count',
'source_top1', 'source_top1_count', 'source_top2', 'source_top2_count',
'source_top3', 'source_top3_count', 'freeze_status', 'notes',
'kept_share_of_raw', 'duplicate_burden', 'dropped_share', 'shock_month_flag',
'shock_window_flag_placeholder']
```

Article date/month-like columns: ['date', 'year', 'month', 'date_raw']

Monthly date/month-like columns: ['month_id', 'year', 'month',
'shock_month_flag']

Duplicate article_id row count (all rows): 0

```
[4]: month_key_candidates = ["month_id", "month", "year_month", "batch_id"]
MONTH_COL = next((c for c in month_key_candidates if c in monthly.columns),
↳None)
if MONTH_COL is None:
    raise ValueError("Could not find a month key column in monthly dataset")

print("Selected MONTH_COL:", MONTH_COL)

monthly_cov = monthly.copy()
monthly_cov["month_key"] = monthly_cov[MONTH_COL].astype(str)
monthly_cov["month_dt"] = pd.to_datetime(monthly_cov["month_key"],
↳format="%Y-%m", errors="coerce")
monthly_cov = monthly_cov.sort_values("month_dt")

observed_months = monthly_cov["month_dt"].dropna().dt.to_period("M")
first_month = str(observed_months.min()) if not observed_months.empty else None
last_month = str(observed_months.max()) if not observed_months.empty else None
observed_count = int(observed_months.nunique())

expected_idx = pd.period_range("2013-01", "2025-12", freq="M")
expected_count = len(expected_idx)
missing_months = [str(p) for p in expected_idx if p not in set(observed_months)]

print("First observed month:", first_month)
print("Last observed month:", last_month)
print("Observed month count:", observed_count)
print("Expected month count:", expected_count)
print("Missing months:", missing_months if missing_months else "none")
```

```
Selected MONTH_COL: month_id
First observed month: 2013-01
Last observed month: 2025-12
Observed month count: 156
Expected month count: 156
Missing months: none
```

```
[5]: count_col_candidates = [
    "n_articles_kept",
    "kept_article_count",
    "kept_count",
    "article_count_kept",
    "n_kept",
    "raw_article_count",
]
COUNT_COL = next((c for c in count_col_candidates if c in monthly.columns),
↳None)
if COUNT_COL is None:
```

```

    possible = [c for c in monthly.columns if "count" in c.lower() or "article" in c.lower()]
    raise ValueError(f"No suitable monthly count column found. Candidate columns were: {possible}")

print("Selected COUNT_COL:", COUNT_COL)

```

Selected COUNT_COL: kept_article_count

```

[6]: vol = monthly_cov.copy()
vol[COUNT_COL] = pd.to_numeric(vol[COUNT_COL], errors="coerce")

print(vol[COUNT_COL].describe())

plt.figure(figsize=(12, 4))
plt.plot(vol["month_dt"], vol[COUNT_COL], linewidth=1.5)
plt.title(f"Monthly Kept Article Count Over Time ({COUNT_COL})")
plt.xlabel("Month")
plt.ylabel("Kept articles")
plt.tight_layout()
plt.savefig(FIGURES_DIR / "monthly_kept_article_count_over_time_report.png", dpi=150)
plt.show()
plt.close()

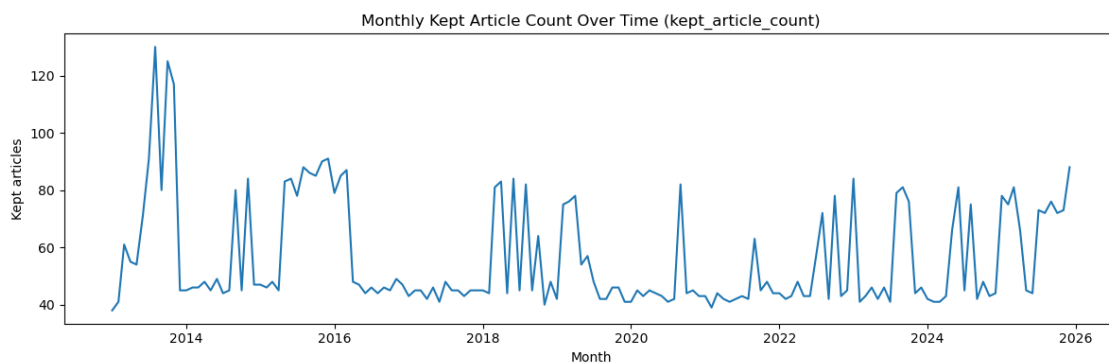
print("Saved:", FIGURES_DIR / "monthly_kept_article_count_over_time_report.png")

```

```

count      156.000000
mean        56.230769
std         18.790473
min         38.000000
25%         43.000000
50%         46.000000
75%         73.000000
max         130.000000
Name: kept_article_count, dtype: float64

```



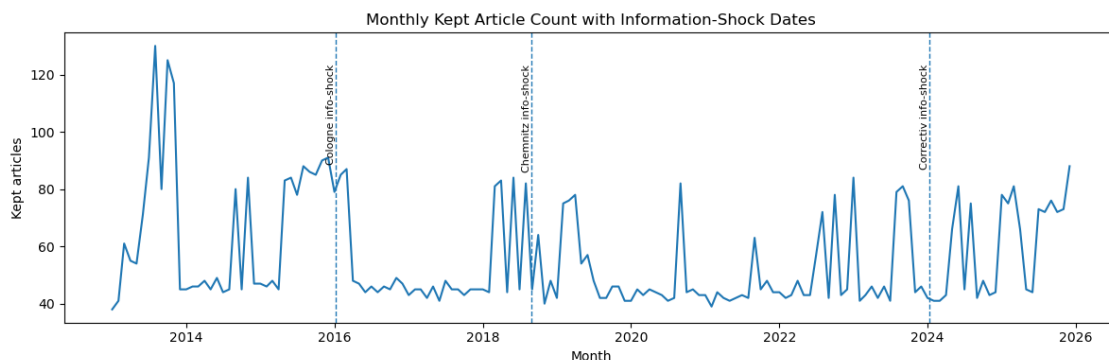
Saved: C:\PythonProjects\AfD-project\figures\01_notebook_figures\monthly_kept_article_count_over_time_report.png

```
[7]: shock_dates = {
    "Cologne info-shock": pd.Timestamp("2016-01-05"),
    "Chemnitz info-shock": pd.Timestamp("2018-08-27"),
    "Correctiv info-shock": pd.Timestamp("2024-01-10"),
}

plt.figure(figsize=(12, 4))
plt.plot(vol["month_dt"], vol[COUNT_COL], linewidth=1.5)
for label, dt in shock_dates.items():
    plt.axvline(dt, linestyle="--", linewidth=1)
    plt.text(dt, vol[COUNT_COL].max() * 0.95, label, rotation=90, va="top",
             ha="right", fontsize=8)

plt.title("Monthly Kept Article Count with Information-Shock Dates")
plt.xlabel("Month")
plt.ylabel("Kept articles")
plt.tight_layout()
plt.savefig(FIGURES_DIR / "monthly_salience_with_shock_dates.png", dpi=150)
plt.show()
plt.close()

print("Saved:", FIGURES_DIR / "monthly_salience_with_shock_dates.png")
```



Saved: C:\PythonProjects\AfD-project\figures\01_notebook_figures\monthly_salience_with_shock_dates.png

```
[8]: an = vol[["month_key", "month_dt", COUNT_COL]].copy()
an = an.sort_values("month_dt")
```



```

mean_count = an[COUNT_COL].mean()
std_count = an[COUNT_COL].std(ddof=0)
an["salience_z"] = (an[COUNT_COL] - mean_count) / std_count if std_count and
↳not np.isnan(std_count) else np.nan

print("Mean monthly kept count:", mean_count)
print("Std monthly kept count:", std_count)

print()
print("Top 15 months by kept count:")
display(an.sort_values(COUNT_COL, ascending=False).head(15)[["month_key",
↳COUNT_COL, "salience_z"]])

print()
print("Months with salience_z >= 2.0:")
high_z = an[an["salience_z"] >= 2.0].sort_values("salience_z", ascending=False)
display(high_z[["month_key", COUNT_COL, "salience_z"]])

qc_sensitive = ["2013-08", "2018-11", "2018-12", "2019-01"]
print()
print("QC-sensitive months check:")
display(an[an["month_key"].isin(qc_sensitive)][["month_key", COUNT_COL,
↳"salience_z"]].sort_values("month_key"))

```

Mean monthly kept count: 56.23076923076923

Std monthly kept count: 18.73015057157644

Top 15 months by kept count:

	month_key	kept_article_count	salience_z
7	2013-08	130	3.938528
9	2013-10	125	3.671579
10	2013-11	117	3.244460
6	2013-07	91	1.856324
35	2015-12	91	1.856324
34	2015-11	90	1.802934
155	2025-12	88	1.696155
31	2015-08	88	1.696155
38	2016-03	87	1.642765
32	2015-09	86	1.589375
33	2015-10	85	1.535985
37	2016-02	85	1.535985
22	2014-11	84	1.482595
120	2023-01	84	1.482595
29	2015-06	84	1.482595

Months with salience_z >= 2.0:

	month_key	kept_article_count	salience_z
7	2013-08	130	3.938528
9	2013-10	125	3.671579
10	2013-11	117	3.244460

QC-sensitive months check:

	month_key	kept_article_count	salience_z
7	2013-08	130	3.938528
70	2018-11	40	-0.866558
71	2018-12	48	-0.439440
72	2019-01	42	-0.759779

1.0.1 Duplicate and exclusion burden checks

This is an audit/QC section using monthly exclusion/duplicate fields (not analytical corpus composition).

```
[9]: dup_like_cols = [
      c for c in monthly.columns
      if any(k in c.lower() for k in ["duplicate", "dup", "regional", "drop",
      ↪ "excluded"])]
    ]
    print("Detected duplicate/exclusion-related monthly columns:")
    print(dup_like_cols)

    if dup_like_cols:
        cols = ["month_key", COUNT_COL] + dup_like_cols
        dup_view = vol.copy()
        dup_view["month_key"] = dup_view["month_key"].astype(str)
        display(dup_view[cols].head(12))

        qc_sensitive = ["2013-08", "2018-11", "2018-12", "2019-01"]
        print()
        print("QC-sensitive months duplicate/exclusion view:")
        display(dup_view[dup_view["month_key"].isin(qc_sensitive)][cols].
        ↪ sort_values("month_key"))
    else:
        print("No duplicate/exclusion-related columns found in monthly_summary.
        ↪ This check depends on available columns.")
```

Detected duplicate/exclusion-related monthly columns:

```
['dropped_article_count', 'duplicate_exact_count', 'duplicate_near_count',
'regional_variant_count', 'reader_letter_drop_count',
'commentary_noncore_drop_count', 'very_short_low_value_drop_count',
'duplicate_burden', 'dropped_share']
```

month_key	kept_article_count	dropped_article_count	duplicate_exact_count	↪ duplicate_near_count	regional_variant_count	\
-----------	--------------------	-----------------------	-----------------------	------------------------	------------------------	---

0	2013-01		38		4	1	␣
↩		0		1			
1	2013-02		41		14	2	␣
↩		0		4			
2	2013-03		61		6	0	␣
↩		0		2			
3	2013-04		55		8	1	␣
↩		0		2			
4	2013-05		54		10	0	␣
↩		0		0			
5	2013-06		71		12	0	␣
↩		0		4			
6	2013-07		91		13	1	␣
↩		0		4			
7	2013-08		130		16	2	␣
↩		1		6			
8	2013-09		80		11	2	␣
↩		0		4			
9	2013-10		125		16	1	␣
↩		0		11			
10	2013-11		117		19	9	␣
↩		1		4			
11	2013-12		45		5	1	␣
↩		0		2			

	reader_letter_drop_count	commentary_noncore_drop_count			
↩	very_short_low_value_drop_count	duplicate_burden	dropped_share		
0		1	1		␣
↩	0	0.047619	0.095238		
1		0	8		␣
↩	0	0.109091	0.254545		
2		0	4		␣
↩	0	0.029851	0.089552		
3		1	4		␣
↩	0	0.047619	0.126984		
4		2	8		␣
↩	0	0.000000	0.156250		
5		0	7		␣
↩	0	0.048193	0.144578		
6		2	5		␣
↩	0	0.048077	0.125000		
7		1	5		␣
↩	0	0.061644	0.109589		
8		1	4		␣
↩	0	0.065934	0.120879		

9		0		4	
↪	0	0.085106	0.113475		
10		1		3	
↪	0	0.102941	0.139706		
11		0		2	
↪	0	0.060000	0.100000		

QC-sensitive months duplicate/exclusion view:

	month_key	kept_article_count	dropped_article_count	duplicate_exact_count	
					↪ duplicate_near_count regional_variant_count \
7	2013-08		130	16	2
↪		1	6		
70	2018-11		40	50	45
↪		0	1		
71	2018-12		48	50	41
↪		0	6		
72	2019-01		42	108	103
↪		0	3		

	reader_letter_drop_count	commentary_noncore_drop_count	
			↪ very_short_low_value_drop_count duplicate_burden dropped_share
7		1	5
↪	0	0.061644	0.109589
70		0	4
↪	0	0.511111	0.555556
71		0	3
↪	0	0.479592	0.510204
72		0	2
↪	0	0.706667	0.720000

```
[10]: source_col_candidates = ["publication", "publication_normalized", "source",
↪ "source_name"]
SOURCE_COL = next((c for c in source_col_candidates if c in articles_kept.
↪ columns), None)
print("Selected SOURCE_COL (kept corpus):", SOURCE_COL)

if SOURCE_COL is not None:
    src = articles_kept.copy()
    src[SOURCE_COL] = src[SOURCE_COL].fillna("").astype(str).str.strip()
    src = src[src[SOURCE_COL] != ""]
    top20 = src[SOURCE_COL].value_counts().head(20)
    display(top20.to_frame("article_count"))

    top15 = top20.head(15).sort_values(ascending=True)
    plt.figure(figsize=(8, 6))
```

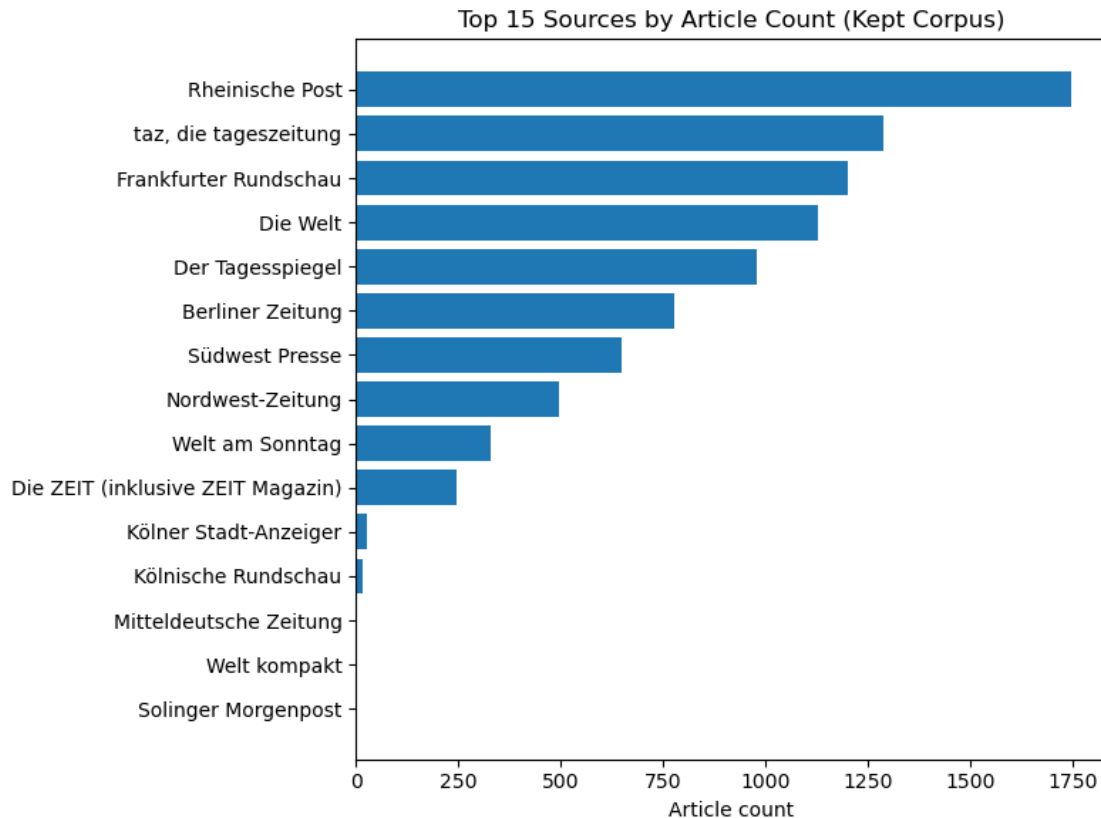
```

plt.barh(top15.index, top15.values)
plt.title("Top 15 Sources by Article Count (Kept Corpus)")
plt.xlabel("Article count")
plt.tight_layout()
plt.savefig(FIGURES_DIR / "top15_sources_report.png", dpi=150)
plt.show()
plt.close()
print("Saved:", FIGURES_DIR / "top15_sources_report.png")
else:
    print("No source/publication column found; skipping source composition_
    ↪plots.")

```

Selected SOURCE_COL (kept corpus): publication

	article_count
publication	
Rheinische Post	1745
taz, die tageszeitung	1288
Frankfurter Rundschau	1200
Die Welt	1128
Der Tagesspiegel	979
Berliner Zeitung	776
Südwest Presse	647
Nordwest-Zeitung	494
Welt am Sonntag	330
Die ZEIT (inklusive ZEIT Magazin)	246
Kölner Stadt-Anzeiger	26
Kölnische Rundschau	15
Welt kompakt	1
Mitteldeutsche Zeitung	1
Solinger Morgenpost	1
Lauterbacher Anzeiger (Germany)	1
Oberhessische Zeitung (Germany)	1
Main-Taunus-Kurier	1
Groß-Gerauer Echo	1
Wiesbadener Tagblatt	1



Saved: C:\PythonProjects\AfD-
project\figures\01_notebook_figures\top15_sources_report.png

```
[11]: length_col_candidates = ["word_count", "body_char_count", "text_length",
    ↪ "article_length"]
LENGTH_COL = next((c for c in length_col_candidates if c in articles_kept.
    ↪ columns), None)
print("Selected LENGTH_COL (kept corpus):", LENGTH_COL)

if LENGTH_COL is not None:
    lengths = pd.to_numeric(articles_kept[LENGTH_COL], errors="coerce").dropna()
    print(lengths.describe(percentiles=[0.25, 0.5, 0.75, 0.9, 0.95, 0.99]))

    if len(lengths) > 0:
        cap = lengths.quantile(0.99)
        plot_vals = lengths[lengths <= cap]
        plt.figure(figsize=(8, 4))
        plt.hist(plot_vals, bins=40)
        plt.title(f"Article Length Distribution ({LENGTH_COL}, kept corpus,
    ↪ clipped at p99)")
        plt.xlabel(LENGTH_COL)
```

```

plt.ylabel("Frequency")
plt.tight_layout()
plt.savefig(FIGURES_DIR / "article_length_distribution_report.png",
↳dpi=150)
plt.show()
plt.close()
print("Saved:", FIGURES_DIR / "article_length_distribution_report.png")
else:
    print("No suitable article length column found.")

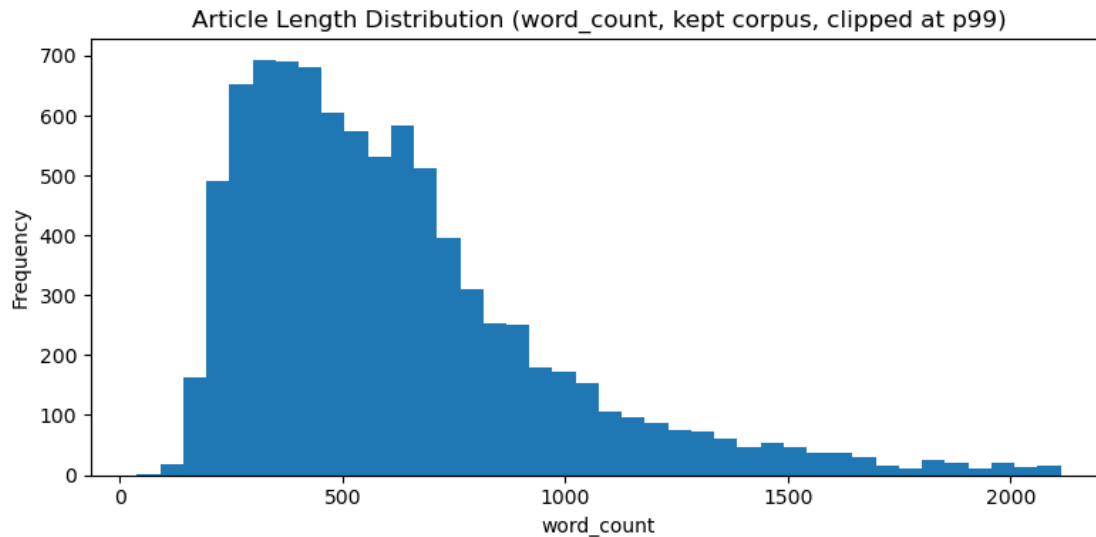
text_col_candidates = ["text_body", "full_text", "body_text", "clean_text"]
TEXT_COL = next((c for c in text_col_candidates if c in articles_kept.columns),
↳None)
print("Selected TEXT_COL (kept corpus):", TEXT_COL)
if TEXT_COL is not None:
    empty_text = articles_kept[TEXT_COL].fillna("").astype(str).str.strip().
↳eq("").sum()
    print("Empty/missing text rows in kept corpus:", int(empty_text), "out of",
↳len(articles_kept))

```

Selected LENGTH_COL (kept corpus): word_count

count	8882.000000
mean	641.985927
std	432.241704
min	38.000000
25%	364.000000
50%	544.500000
75%	776.000000
90%	1115.000000
95%	1423.000000
99%	2114.660000
max	6804.000000

Name: word_count, dtype: float64



Saved: C:\PythonProjects\AfD-
 project\figures\01_notebook_figures\article_length_distribution_report.png
 Selected TEXT_COL (kept corpus): text_body
 Empty/missing text rows in kept corpus: 0 out of 8882

```
[12]: indicator = vol[["month_key", "month_dt", COUNT_COL]].copy()
indicator = indicator.rename(columns={"month_key": "month", COUNT_COL:
    ↳ "n_articles_kept"})
indicator = indicator.sort_values("month_dt").drop_duplicates(subset=["month"])

indicator["year_month"] = indicator["month"]
indicator["year"] = indicator["month_dt"].dt.year
indicator["month_num"] = indicator["month_dt"].dt.month
indicator["media_salience_log1p"] = np.log1p(indicator["n_articles_kept"])

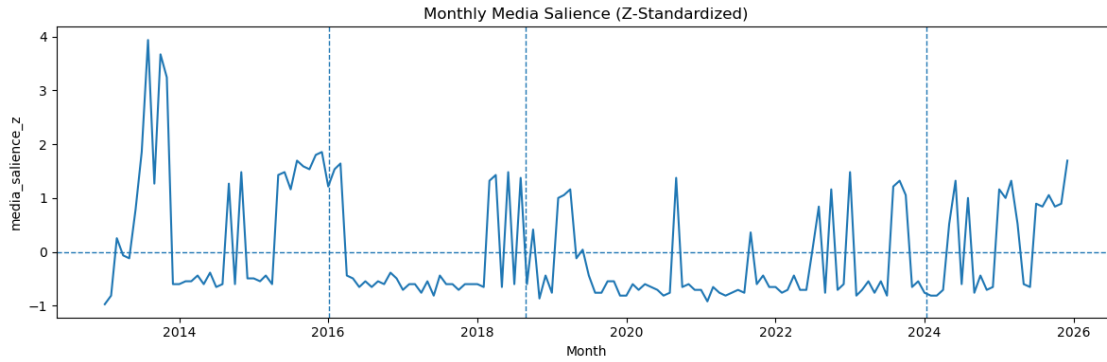
sal_mean = indicator["n_articles_kept"].mean()
sal_std = indicator["n_articles_kept"].std(ddof=0)
indicator["media_salience_z"] = (indicator["n_articles_kept"] - sal_mean) /
    ↳ sal_std if sal_std and not np.isnan(sal_std) else np.nan

plt.figure(figsize=(12, 4))
plt.plot(indicator["month_dt"], indicator["media_salience_z"], linewidth=1.5)
plt.axhline(0, linestyle="--", linewidth=1)
for label, dt in shock_dates.items():
    plt.axvline(dt, linestyle="--", linewidth=1)
plt.title("Monthly Media Salience (Z-Standardized)")
plt.xlabel("Month")
plt.ylabel("media_salience_z")
plt.tight_layout()
```



```
plt.savefig(FIGURES_DIR / "monthly_media_salience_z_report.png", dpi=150)
plt.show()
plt.close()

print("Saved:", FIGURES_DIR / "monthly_media_salience_z_report.png")
display(indicator.head(12))
```



Saved: C:\PythonProjects\AfD-project\figures\01_notebook_figures\monthly_media_salience_z_report.png

	month	month_dt	n_articles_kept	year_month	year	month_num	
	media_salience_log1p	media_salience_z					
0	2013-01	2013-01-01	38	2013-01	2013	1	
	3.663562	-0.973338					
1	2013-02	2013-02-01	41	2013-02	2013	2	
	3.737670	-0.813169					
2	2013-03	2013-03-01	61	2013-03	2013	3	
	4.127134	0.254629					
3	2013-04	2013-04-01	55	2013-04	2013	4	
	4.025352	-0.065711					
4	2013-05	2013-05-01	54	2013-05	2013	5	
	4.007333	-0.119100					
5	2013-06	2013-06-01	71	2013-06	2013	6	
	4.276666	0.788527					
6	2013-07	2013-07-01	91	2013-07	2013	7	
	4.521789	1.856324					
7	2013-08	2013-08-01	130	2013-08	2013	8	
	4.875197	3.938528					
8	2013-09	2013-09-01	80	2013-09	2013	9	
	4.394449	1.269036					
9	2013-10	2013-10-01	125	2013-10	2013	10	
	4.836282	3.671579					
10	2013-11	2013-11-01	117	2013-11	2013	11	
	4.770685	3.244460					

11	2013-12	2013-12-01	45	2013-12	2013	12	
	↳3.828641	-0.599609					

```
[13]: merge_ready = indicator[[
        "month",
        "year_month",
        "year",
        "month_num",
        "n_articles_kept",
        "media_salience_log1p",
        "media_salience_z",
    ]].copy()

merge_ready = merge_ready.sort_values("month").
↳drop_duplicates(subset=["month"], keep="first")

out_csv = DATA_DIR / "monthly_media_salience_indicator.csv"
out_parquet = DATA_DIR / "monthly_media_salience_indicator.parquet"

merge_ready.to_csv(out_csv, index=False, encoding="utf-8-sig")
try:
    merge_ready.to_parquet(out_parquet, index=False)
    wrote_parquet = True
except Exception as exc:
    wrote_parquet = False
    print("Parquet write skipped:", exc)

print("Rows in merge-ready indicator:", len(merge_ready))
print("Saved:", out_csv)
if wrote_parquet:
    print("Saved:", out_parquet)

display(merge_ready.tail(12))
```

Rows in merge-ready indicator: 156

Saved: C:\PythonProjects\AfD-project\data\monthly_media_salience_indicator.csv

Saved: C:\PythonProjects\AfD-project\data\monthly_media_salience_indicator.parquet

	month	year_month	year	month_num	n_articles_kept	media_salience_log1p	↳media_salience_z
144	2025-01	2025-01	2025	1	78	4.369448	↳1.162256
145	2025-02	2025-02	2025	2	75	4.330733	↳1.002086
146	2025-03	2025-03	2025	3	81	4.406719	↳1.322426

147	2025-04	2025-04	2025	4	66	4.204693	↵
	↵	0.521578					
148	2025-05	2025-05	2025	5	45	3.828641	↵
	↵	-0.599609					
149	2025-06	2025-06	2025	6	44	3.806662	↵
	↵	-0.652999					
150	2025-07	2025-07	2025	7	73	4.304065	↵
	↵	0.895307					
151	2025-08	2025-08	2025	8	72	4.290459	↵
	↵	0.841917					
152	2025-09	2025-09	2025	9	76	4.343805	↵
	↵	1.055476					
153	2025-10	2025-10	2025	10	72	4.290459	↵
	↵	0.841917					
154	2025-11	2025-11	2025	11	73	4.304065	↵
	↵	0.895307					
155	2025-12	2025-12	2025	12	88	4.488636	↵
	↵	1.696155					

1.1 Interpretation Notes and Next Steps

- This indicator (`n_articles_kept`, `media_salience_log1p`, `media_salience_z`) measures media salience/discourse intensity, not public opinion.
- Peaks should be interpreted with substantive context and QC context.
- The next stage is preparing AfD polling data at monthly frequency and merging on stable monthly keys (`month`, `year_month`).
- Only after polling merge should preliminary event-study style or lag-descriptive checks be considered.